ELECTRONIC PAYMENT GATEWAY

Complete Pipeline Technical Documentation

Database	MySQL 8.0 (payment_gateway)
CDC Engine	Debezium 2.4 + Kafka Connect
Message Broker	Apache Kafka 3.4 (cp-kafka 7.4.1)
Object Storage	MinIO — Parquet (Snappy)
Transforms	dbt 1.7 (staging + marts)
Orchestration	Apache Airflow 2.9.3
Analytics DB	PostgreSQL 15 (analytics schema)
Language	Python 3.11
Date	March 18, 2026

Table of Contents

1. Architecture Overview

The payment gateway is a fully containerised data platform. Data originates in MySQL, is captured in real time by Debezium Change Data Capture, streamed through Apache Kafka, written to MinIO as Parquet files, transformed by dbt, loaded into PostgreSQL, and orchestrated end-to-end by Apache Airflow.

```
DATA GENERATION
  generate_data.py  -> 155k txns + 10k clients + fraud patterns (once)
  stream_data.py    -> 2 txns/sec + 1 fraud/hr (continuous)
        | v
MYSQL 8.0  (payment_gateway -- 13 tables, binlog ROW format)
        | binlog stream | v
DEBEZIUM  (Kafka Connect 2.4 -- snapshot + CDC)
  snapshot.mode=initial  ->  all 13 tables on first run
  binlog stream  ->  INSERT / UPDATE / DELETE in < 1 second
        | v
KAFKA TOPICS  (cdc.payment_gateway.*)
        | v
KAFKA CONSUMER  -> Parquet (Snappy) -> MinIO
  s3://payment-gateway/raw/{table}/year=/month=/day/  (Hive partitioned)
        | v
DBT  (staging views + 4 mart tables in MySQL)
        | v
AIRFLOW DAG  ->  load_marts_to_postgres  ->  PostgreSQL analytics
```

1.1 Docker Services

Service	Image	Port	Role
MySQL 8.0	mysql:8.0	3306	Transactional DB — binlog CDC source
Adminer	adminer:4.8.1	8081	Web SQL browser
Zookeeper	cp-zookeeper:7.4.1	2181	Kafka coordination
Kafka	cp-kafka:7.4.1	9092	Message broker
Kafka Connect	debezium/connect:2.4	8083	Debezium CDC engine
Kafdrop	kafdrop:latest	9000	Kafka topic browser UI
MinIO	minio:latest	9001/9002	S3-compatible object storage
data_seeder	custom	—	One-shot bulk data generator (155k rows)
data_streamer	custom	—	Live data generator (2 txns/sec)
kafka_consumer	custom	—	Debezium CDC → Parquet → MinIO
PostgreSQL	postgres:15	5432	Airflow metadata + analytics DB
Airflow	custom airflow:2.9.3	8082	DAG orchestration with dbt

2. MySQL Database

2.1 Schema — 13 Tables

Table	Category	Rows (seeded)	Description
countries	Reference	10	ISO country codes + default currencies
transaction_types	Reference	7	PURCHASE, REFUND, WITHDRAWAL, etc.
transaction_statuses	Reference	7	APPROVED, DECLINED, SETTLED, etc.
card_networks	Reference	4	Visa, Mastercard, Amex, Discover
merchant_categories	Reference	15	ISO MCC codes
merchants	Merchant	200	Registered businesses
pos_terminals	Merchant	1,000+	Physical and virtual terminals
clients	Client	10,000+	Registered cardholders
payment_cards	Client	15,000-25,000	Tokenised PAN instruments
transactions	Transaction	155,000+	Core payment events (all channels)
fraud_alerts	Risk	~31,000+	Fraud detection rules per transaction
refunds	Risk	~7,500+	Refund and chargeback records
settlement_batches	Settlement	73,000	Daily merchant payout aggregates

2.2 Binlog Configuration (CDC Requirements)

MySQL is started with these flags to enable Debezium row-level change capture:

Flag	Value	Purpose
--server-id	1	Unique server ID in replication topology
--log-bin	mysql-bin	Enable binary logging
--binlog-format	ROW	Row-level changes (not statement-level)
--binlog-row-image	FULL	Full before/after row images in binlog
--expire-logs-days	3	Auto-purge binlog after 3 days

2.3 Debezium MySQL Privileges

The pg_user account has these privileges granted via 00_debezium_grants.sql:

Privilege	Why Debezium Needs It
SELECT ON *.*	Read all tables during initial snapshot

Privilege	Why Debezium Needs It
RELOAD	Execute FLUSH TABLES WITH READ LOCK during snapshot — required since Debezium 2.x
SHOW DATABASES	Enumerate databases to locate payment_gateway
REPLICATION SLAVE	Connect as replication client to stream the binlog
REPLICATION CLIENT	Query binlog position via SHOW MASTER STATUS
SELECT ON performance_schema.*	Read replication metrics and status

3. Debezium CDC

3.1 How Debezium Works

Debezium is deployed as a Kafka Connect connector. It acts as a MySQL replication slave — MySQL sends it every binlog event in real time. On first start it takes a full snapshot of all configured tables (equivalent to the old kafka_producer full load), then continuously streams INSERT, UPDATE, and DELETE events.

Phase	What Happens	Latency
Initial Snapshot	Reads every row from all 13 tables, publishes to cdc.* topics	Minutes (one-time)
Binlog Streaming	Captures every INSERT/UPDATE/DELETE as a Kafka message	< 1 second

3.2 CDC Topics

One topic per table, named: cdc.payment_gateway.{table}

```
cdc.payment_gateway.transactions
cdc.payment_gateway.clients
cdc.payment_gateway.merchants
cdc.payment_gateway.pos_terminals
cdc.payment_gateway.payment_cards
cdc.payment_gateway.fraud_alerts
cdc.payment_gateway.refunds
cdc.payment_gateway.settlement_batches
cdc.payment_gateway.countries
cdc.payment_gateway.transaction_types
cdc.payment_gateway.transaction_statuses
cdc.payment_gateway.card_networks
cdc.payment_gateway.merchant_categories
```

3.3 Message Format

After the ExtractNewRecordState SMT unwraps the Debezium envelope, each message is a flat row dict with CDC metadata fields prefixed by __:

```
{
  "txn_id":      1042,
  "amount":      142.50,
  "currency":    "USD",
  "is_flagged":  0,
  "initiated_at":"2025-03-14T13:45:22",
  ...

  "__op":           "c",             // r=snapshot  c=insert  u=update  d=delete
  "__source_ts_ms": 1741954000000,   // binlog event timestamp (ms)
  "__table":        "transactions",
  "__deleted":      "false"          // "true" for DELETE events
}
```

3.4 Connector Configuration

Config Key	Value	Notes
snapshot.mode	initial	Full snapshot on first run, then binlog only
snapshot.locking.mode	minimal	Minimise table lock duration during snapshot
decimal.handling.mode	double	Convert DECIMAL to float64 (Parquet compatible)
transforms	unwrap (ExtractNewRecordState)	Flatten Debezium envelope to plain row
tombstones.on.delete	true	Send null-value tombstone after DELETE for log compaction
heartbeat.interval.ms	10000	Keep binlog position fresh when tables are idle

4. Data Generation

4.1 Bulk Seeder (generate_data.py)

Runs once on first deployment. Includes an idempotency guard — exits 0 immediately if transactions already has >= 1,000 rows.

Entity	Volume	Notes
Merchants	200	95% active, all countries and MCC codes
POS Terminals	1,000	97% active, ~5 per merchant
Clients	10,000	96% active, registered 1-2 years prior
Payment Cards	15,000-25,000	1-4 per client, one default
Transactions	155,000+	80% normal, 20% fraud (6 patterns)
Fraud Alerts	~31,000+	1-3 rules per flagged txn, 70% reviewed
Refunds	~5% of approved	30% partial refunds
Settlement Batches	73,000	200 merchants x 365 days

4.2 Streamer (stream_data.py)

Event	Rate	Description
Normal transaction	2 / second	All channels and statuses
Fraud campaign	1 / hour	Random pattern: velocity burst, card testing, high-value, night-owl, international, ATO
New client	2 / hour	New cardholder + 1-3 cards, live cache updated
New POS terminal	1 / day	New terminal for random merchant

5. Kafka Consumer (CDC → MinIO Parquet)

Subscribes to all cdc.payment_gateway.* topics and writes Snappy-compressed Parquet files to MinIO using Hive-style date partitioning.

5.1 MinIO Layout

```
s3://payment-gateway/
  raw/
    transactions/
      year=2025/month=03/day=14/
        20250314_153022_abc123.parquet   (1,000 rows, ~180 KB)
    clients/  merchants/  fraud_alerts/  ...
```

5.2 CDC Event Handling

Event Type	__op	Action
Snapshot row	r	Buffer row → flush to Parquet
INSERT	c	Buffer row → flush to Parquet
UPDATE	u	Buffer updated row → flush to Parquet
DELETE	d	Skip by default (SKIP_DELETES=true)
Tombstone	null value	Always skip (Kafka compaction marker)

5.3 Flush Strategy

Condition	Value	Rationale
Row threshold	1,000 rows	Prevent memory growth, right-sized Parquet files
Time threshold	30 seconds	Guarantee files appear in MinIO regularly
Background timer	Checks every 5s	Catch low-traffic tables
Shutdown	Immediate on SIGTERM	No data loss on graceful stop

5.4 Reading Parquet Files

```
# Pandas
import pandas as pd
df = pd.read_parquet(
    "s3://payment-gateway/raw/transactions/year=2025/month=03/day=14/",
    storage_options={"endpoint_url": "http://localhost:9002",
                     "key": "minioadmin", "secret": "minioadmin123"}
)

# DuckDB
import duckdb
```

```
duckdb.sql("""
    SELECT channel, COUNT(*) AS txns, SUM(amount) AS revenue
    FROM read_parquet('s3://payment-gateway/raw/transactions/**/*.parquet')
    GROUP BY channel
""").show()
```

6. dbt Transformations

6.1 Two-Layer Model

Layer	Materialisation	Purpose
staging	VIEW	Type-cast and clean raw MySQL tables (stg_transactions, stg_clients, stg_merchants, stg_fraud_alerts)
marts	TABLE	Aggregated analytics-ready tables built from staging

6.2 Mart Models

Model	Refreshed	Key Metrics
mart_daily_transaction_summary	Daily 02:00	Volume, revenue, fees, fraud rate by date/channel/currency
mart_merchant_performance	Daily 02:00	Gross revenue, net payout, fraud rate per merchant
mart_client_360	Daily 02:00	Total spend, card count, approved/declined/fraud ratios per client
mart_fraud_analysis	Daily 02:00	All fraud alerts with transaction + client context, alert outcome label

7. Airflow Orchestration

7.1 DAGs

DAG	Schedule	What It Does
dbt_hourly_staging	Every hour at :05	Refresh all 4 staging views, run staging tests
dbt_daily_marts	Daily 02:00 UTC	Full run: staging → 4 marts in parallel → all tests → docs generate
load_marts_to_postgres	Daily 03:00 UTC	TRUNCATE + reload all 4 mart tables from MySQL into PostgreSQL analytics DB
pipeline_monitor	Every 15 minutes	Check MySQL health, ingestion rate, fraud spike, unreviewed alert backlog, dbt source freshness

7.2 dbt_daily_marts Task Graph

```
dbt_debug
    │
dbt_run_staging
    │
    ├── mart_daily_transaction_summary ─┐
    ├── mart_merchant_performance       ├──▶ dbt_test_all → dbt_docs_generate
    ├── mart_client_360                 │
    └── mart_fraud_analysis ────────────┘
```

7.3 PostgreSQL Analytics Database

The same PostgreSQL container hosts two databases:

Database	Purpose	Created By
${POSTGRES_DB}	Airflow metadata (DAG state, task logs)	POSTGRES_DB env var
analytics	dbt mart tables for downstream BI/analytics	postgres-init/02_analytics_schema.sql

8. Boot Sequence

```
Zookeeper     → healthy (cub zk-ready)
      |
Kafka         → healthy (cub kafka-ready)
      |
MySQL         → healthy (mysqladmin ping)
      |                  ↓ runs 00_debezium_grants.sql + init.sql
data_seeder   → completed (155k rows inserted)
      |
Kafka Connect → healthy (curl /connectors)
      |
debezium-register → POST connector → exits 0
      |
Debezium: snapshot all 13 tables → then streams binlog
      |
MinIO         → healthy
      |
kafka_consumer → subscribes cdc.payment_gateway.* → Parquet → MinIO
PostgreSQL     → healthy → Airflow init → scheduler + webserver
```

8.1 Healthchecks

Service	Tool	Checks
MySQL	mysqladmin ping	MySQL process accepts connections
Zookeeper	cub zk-ready	ZooKeeper ruok/imok application protocol
Kafka	cub kafka-ready	Full broker metadata round-trip
Kafka Connect	curl /connectors	REST API responds with 200
MinIO	curl /minio/health/live	MinIO HTTP health endpoint
PostgreSQL	pg_isready	PostgreSQL accepts connections

9. Troubleshooting

Symptom	Root Cause	Fix
RELOAD privilege error from Debezium	pg_user lacked RELOAD — required for FLUSH TABLES during snapshot	Add to 00_debezium_grants.sql, then: docker compose down -v && up --build
payment_gateway_db unhealthy	Combined mysqladmin + SQL healthcheck fails inside container shell due to quoting/path issues	Use only mysqladmin ping in healthcheck; let entrypoint.py handle schema check
No cdc.* topics in Kafdrop	debezium-register fired before data_seeder completed — snapshot ran on empty DB	Fixed: debezium-register depends_on data_seeder: service_completed_successfully
kafka_consumer exits: no topics	Consumer started before Debezium created topics	Fixed: kafka_consumer depends_on debezium-register: service_completed_successfully
[Kafka] Attempt N/90 forever	bash not installed in python:3.11-slim — /bin/bash shebang failed silently	Fixed: entrypoint_pipeline.py — pure Python, no shell dependency
Kafka broker crashes on startup	Duplicate PLAINTEXT:// listener names	Fixed: single PLAINTEXT listener only
Zookeeper logs socket closed spam	Healthcheck opened raw TCP without sending ZK protocol	Fixed: cub zk-ready + ZOOKEEPER_4LW_COMMANDS_WHITELIST =ruok